Les structures de données en Python

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Les structures de données de base **4**

A. Introduction 4

B. Les listes 4

C. Les tableaux 8

D. Les dictionnaires 11

III - Structure de données avancées **14**

A. Les tuples 14

B. Les piles 15

C. Les files 16

IV - Structures spécialisées **18**

A. Les graphes 18

B. Les arbres 20

V - Essentiel **24**

VI - Auto-évaluation **25**

A. Exercice 25

B. Test 25

Solutions des exercices **29**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 60 minutes

Matériel : IDE

Contexte

Les structures de données sont essentielles en informatique pour organiser et gérer les informations efficacement. Par exemple, un carnet d'adresses peut être optimisé en utilisant des dictionnaires pour accéder rapidement aux contacts par leur nom, plutôt que de parcourir une liste entière. Une file d'attente à la caisse d'un supermarché illustre bien le fonctionnement d'une file (queue) en informatique, où les clients sont servis dans l'ordre d'arrivée (premier arrivé, premier servi). Les piles (stacks) sont utiles pour des actions annulables, comme dans un éditeur de texte où chaque modification peut être annulée dans l'ordre inverse de leur réalisation (LIFO). Les graphes modélisent les réseaux de transport pour trouver les chemins les plus courts entre deux points. Enfin, les arbres structurent les systèmes de fichiers en hiérarchies. Comprendre et maîtriser ces structures permet d'écrire des programmes plus performants et organisés, adaptés à une grande variété de problèmes.

Tous droits réservés à STUDI - Reproduction interdite

II Les structures de données de base

A. Introduction

Az Définition

Les structures de données sont des moyens d'organiser et de stocker des données de manière à pouvoir les utiliser efficacement. En Python, il existe plusieurs types de structures de données de base qui sont essentielles pour tout programmeur. Ces structures de données de base comprennent les listes, les tableaux, les tuples et les dictionnaires. Chacune de ces structures a ses propres caractéristiques et utilisations spécifiques.

Dans cette partie, nous allons explorer ces structures de données de base en détail. Nous commencerons par les listes, qui sont parmi les structures de données les plus utilisées en Python.

B. Les listes

Az Définition

Les listes en Python sont des collections modifiables qui peuvent contenir des éléments de différents types (entiers, chaînes, flottants, objets, etc.). Elles sont définies en utilisant des crochets « *[]* » et les éléments sont séparés par des virgules. La liste est la structure de données la plus utilisée en Python ; elle est très simple à utiliser et puissante.

Pour créer une liste, nous pouvons placer une série d'éléments entre crochets. Pour accéder aux éléments, nous allons utiliser les indices.

```
1 #Création d'une liste
2 fruits = ["pomme", "banane", "cerise"]
3 # Accès aux éléments de la liste
4 print(fruits[0]) # Sortie : pomme
5 print(fruits[2]) # Sortie : cerise
```

Pour accéder au premier élément de cette liste, il suffit de mentionner le nom de la variable, puis, d'utiliser l'indice. Dans les structures de données, Python compte ses index à partir de 0, donc pomme est à l'indice 0, banane à l'indice 1 et cerise à l'indice 2.

Nous pouvons également utiliser un typage static afin d'éviter un typage dynamique.

```
1 #Création d'une liste
2 Fruits: list = ["pomme", "banane", "cerise"]
3 # Accès aux éléments de la liste
4 print(fruits[0]) # Sortie : pomme
5 print(fruits[2]) # Sortie : cerise
```

Jusqu'à présent, nous avons utilisés des indices positifs, mais, nous pouvons utiliser des indices négatifs. Au lieu de compter les indices depuis le début de la liste, nous allons compter depuis la fin, jusqu'au début.

Tous droits réservés à STUDI - Reproduction interdite

```
1 # Création d'une liste
2 fruits: list = ["pomme", "banane", "cerise"]
3 # Accès aux éléments de la liste
4  print(fruits[0]) # Sortie : pomme
5 print (fruits [2]) # Sortie : cerise
6 print(fruits[-1]) # Sortie : cerise
7 print (fruits[-2]) # Sortie : banane
```

Nous allons être plus précis lors de la déclaration de fruit. Nous savons à l'avance que notre liste contient uniquement des chaînes de caractères. Dans ce cas, nous pouvons encore mieux typer la variable. Ajouter entre crochet le type de contenu que possède une liste permet de spécialiser la liste sur un seul type de données, sans cela, nous pouvons mettre toutes les données que l'on souhaite (peu importe leur type).

```
1 # Création d'une liste
2 fruits: list[str] = ["pomme", "banane", "cerise"]
3 # Accès aux éléments de la liste
4  print(fruits[0]) # Sortie : pomme
5 print (fruits [2]) # Sortie : cerise
6 print(fruits[-1]) # Sortie : cerise
7 print (fruits[-2]) # Sortie : banane
```

Déclarer que la liste contient uniquement des chaînes de caractères, n'est pas inutile, cela permet plus tard dans notre application d'être sûr de n'avoir que des chaînes de caractères à l'intérieur.

Utiliser cette écriture augmente la sécurité ainsi que la fiabilité de votre application.

Très souvent, les listes ne sont pas utilisées seules, mais avec des boucles qui sont essentielles pour les parcourir et les manipuler. Les boucles les plus utilisées sont « *for* » et « *while* » pour itérer sur les éléments.

```
1 # Création d'une liste
2 fruits: list[str] = ["pomme", "banane", "cerise"]
3 # Boucle for pour parcourir une liste
4 for fruit in fruits:
5 print(fruit)
6 # Sortie :
7 # pomme
8 # banane
9 # cerise
10 # Boucle while pour parcourir une liste
11 i = 0
12 while i<len(fruits):
13 print(fruits[i])
14 i += 1
15 # Sortie :
16 # pomme
17 # banane
18 # cerise
```

Dans cet exemple, nous avons utilisé la méthode `len()`, qui est également utilisée pour évaluer la longueur des chaînes de caractères.

La boucle for va prendre chaque élément de la liste et les afficher en console via l'instruction `print`. La boucle `while`, quant à elle, va effectuer un traitement tant que nous n'avons pas atteint la fin de la liste. Ensuite, à la fin, elle va incrémenter notre compteur afin de changer d'indice.

Tous droits réservés à STUDI - Reproduction interdite

Fondamental

Les boucles ne sont pas les seuls moyens de parcourir et exploiter des listes. Elles possèdent plusieurs méthodes intégrées qui facilitent leur manipulation. Voici quelques-unes des méthodes les plus couramment utilisées :

- « *append(x)* » : ajoute un élément à la fin de la liste.
- « *insert(i, x)* » : insère un élément à une position donnée.
- « *remove(x)* » : supprime le premier élément avec la valeur donnée.
- « *pop([i])* » : supprime l'élément à la position donnée et le renvoie. Si aucun indice n'est spécifié, « *pop()* » supprime et renvoie le dernier élément de la liste.
- « *index(x[, start[, end]])* » : renvoie l'indice du premier élément avec la valeur donnée.
- « *count(x)* » : renvoie le nombre d'éléments avec la valeur donnée.
- « *sort(key=None, reverse=False)* » : trie les éléments de la liste en place.
- « *reverse()* » : inverse les éléments de la liste en place.
- « *clear()* » : supprime tous les éléments de la liste.

Exemple

Voyons ensemble comment vous pouvez les utiliser.

```
1 fruits: list[str] = ["apple", "pear", "orange", "banane"]
2 # Ajouter des éléments à la liste
3 fruits.append("orange")
4 print(fruits) # Sortie : ['pomme', 'banane', 'cerise', 'orange']
5 # Insérer un élément à une position spécifique
6 fruits.insert(1, "kiwi")
7 print(fruits) # Sortie : ['pomme', 'kiwi', 'banane', 'cerise', 'orange']
8 # Supprimer un élément de la list
9 fruits.remove("banane")
10 print(fruits) # Sortie : ['pomme', 'kiwi', 'cerise', 'orange']
11 # Trier la liste
12 fruits.sort()
13 print(fruits) # Sortie : ['cerise', 'kiwi', 'orange', 'pomme']
14 # Inserver la liste
15 fruits.reverse()
16 print(fruits) # Sortie : ['pomme', 'orange', 'kiwi', 'cerise']
```

Cas pratique : Créer une liste de fruit

José possède un petit magasin de fruits. Il souhaite un système pour gérer efficacement son stock de fruits. José reçoit régulièrement de nouvelles livraisons de fruits qu'il doit ajouter à sa liste de stock. Puis, lorsqu'un fruit est vendu, il doit le retirer de sa liste.

Nous devons permettre à José d'afficher tous les fruits que possède son magasin et de les trier.

Maintenant que vous connaissez un peu mieux les listes en Python, vous proposez à José une solution adaptée à son besoin grâce à vos nouvelles connaissances !

Dans un premier temps, vous allez construire votre liste avec quelques fruits.

```
1 # Liste initiale de fruits
2 stock_fruits: list[str] = ["pomme","banane","cerise"]
```


Tous droits réservés à STUDI - Reproduction interdite

L'ajout de fruits peut être effectué via la méthode append(), qui vous permet d'ajouter un élément. Cependant, José peut avoir besoin d'ajouter plusieurs éléments à sa liste. Par conséquent, vous pouvez également utiliser la méthode extend(), qui permet d'ajouter plusieurs éléments à la fois dans une liste.

```
1 # liste initiale de fruits
2 stock_fruits:list[str] = ["pomme", "banane","cerise"]
3 #Ajouter des fuits
4 stock_fruits.append("orange")
5 #nous pouvons utiliser le méthode extend qui permet d'ajouter plusieurs elements.
6 stock_fruits.extend(["kiwi", "litchi"])
7 print(stock_fruits) #Sortie : ['pomme", "banane", "cerise", "orange", "kiwi',
  'litchi']
```

Pour la suppression, vous pouvez utiliser la méthode remove(), mais elle supprime uniquement le premier élément de la liste qui correspond à l'argument donné. Vous devez donc l'utiliser avec précaution. Par exemple, vous allez supprimer le fruit « *banane* » de la liste.

```
1 # liste initiale de fruits
2 stock_fruits:list[str] = ["pomme","banane","cerise"]
3
4 #Ajouter des fruits
5 stock_fruits.append("orange")
6 # nous pouvons utiliser le méthode extend qui permet d'ajouter plusieurs elements.
  print(stock_fruits)
7 stock_fruits.extend(["kiwi", "litchi"])
8 print(stock_fruits) #Sortie : ['pomme", "banane", "cerise", "orange", "kiwi',
  'litchi']
9 # Retirer des fruits vendus
10 stock_fruits.remove("banane")
11 print(stock_fruits) # Sortie : ['pomme", "cerise", "orange", "kiwi', 'litchi']
```

José souhaite trier la liste, puis l'afficher. Pour cela, la méthode sort() va vous aider. Par défaut, elle trie les données par ordre croissant. L'affichage sera effectué avec la boucle for : vous allez itérer sur chaque élément de la liste afin de l'afficher en console via print().

```
1 # Liste initiale de fruits
2 stock_fruits: list[str] = ["pomme", "banane", "cerise"]
3
4 # Ajouter des fruits
5 stock_fruits.append("orange")
6
7 # nous pouvons utiliser la méthode extend qui permet d'ajouter plusieurs elements.
8 stock_fruits.extend(["kiwi", "litchi"])
9 print(stock_fruits) # sortie : ['pomme', 'banane', 'cerise', 'orange', 'kiwi',
  'litchi']
10
11 # Retirer des fruits vendus
12 stock_fruits.remove("banane")
13 print(stock_fruits) # Sortie : ['pomme', 'cerise', 'orange', 'kiwi', 'litchi']
14
15 #Trier la liste des fruits
16 stock_fruits.sort()
17 print(stock_fruits) # Sortie : ['cerise', 'kiwi', 'litchi', 'orange', 'pomme']
18
19 #Imprimer la liste des fruits
20 for fruit in stock_fruits:
21     print(fruit)
```

Tous droits réservés à STUDI - Reproduction interdite

C. Les tableaux

Az Définition

Les tableaux (array), contrairement aux listes, nécessitent que tous les éléments soient du même type, ce qui peut offrir des avantages en termes de performance et d'utilisation de la mémoire.

En Python, les tableaux peuvent être créés en utilisant le module array de la bibliothèque standard.

Pour créer un tableau, vous devez importer le module array et spécifier le type d'éléments qu'il va contenir. Le type est défini par un code de type (comme 'i' pour les entiers, ou 'd' pour les nombres flottants). Nous pouvons accéder aux données du tableau en utilisant, tout comme pour les listes, les indices.

```
1 import array
2 # Création d'un tableau d'entiers
3 scores = array.array('i', [90, 85, 88, 92])
4 # Accès aux éléments du tableau
5 print(scores[0]) # Sortie : 90
6 print(scores [2]) # Sortie : 88
```

Si vous ajoutez dans votre tableau une chaîne de caractères, en apparence Python aime cela car PyCharm ne vous communique pas d'erreur.

```
1 import array
2 # Création d'un tableau d'entiers
3 scores = array.array('i', [90, 85, 88, 92, 'a' ])
4 # Accès aux éléments du tableau
5 print(scores[0]) # Sortie : 90
6 print(scores [2]) # Sortie : 88
```

Mais lors de l'exécution, une erreur est levée « *TypeError* », ce qui signifie que vous avez utilisé un type de variable qui n'est pas autorisé.

```
/Users/Shared/pythonProject2/.venv/bin/python /Users/Shared/pythonProject2/main.py
Traceback (most recent call last):
  File "/Users/Shared/pythonProject2/main.py", line 3, in <module>
    scores = array.array('i', [90, 85, 88, 92, 'a' ])
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'str' object cannot be interpreted as an integer

Process finished with exit code 1
```

Figure 1 : erreur d'exécution, une string a été mis dans un tableau d'entier

Votre tableau étant déclaré et initialisé, vous pouvez exploiter les résultats. Pour ce faire, il s'agit de la même chose que les listes.

```
1 import array
2
3 # Création d'un tableau d'entiers
4 scores = array.array('i', [98, 85, 88, 92])
5 # Boucle for pour parcourir un tableau
6 for score in scores:
7     print(score)
8
9 # Boucle while pour parcourir un tableau
10 i = 0
```


Tous droits réservés à STUDI - Reproduction interdite

```
while i < len(scores):
    print(scores[i])
i += 1
```

Fondamental

Les méthodes pour exploiter les tableaux sont les mêmes que pour les listes. Dans ce cas, on peut légitimement se poser la question : quand dois-je préférer les tableaux aux listes ?

Vous pouvez dresser un tableau comparatif ! (Par exemple, pour un comparatif, on préfère les tableaux.)

Élément comparatif	Liste	Tableau
Type de données	Peut stocker différent type de données.	Stock un seul type de données qui doit être défini lors de la création.
Performance	Flexible, peu efficace en termes de mémoire et stockage de gros volume de données.	Efficace en termes de gestion de mémoire. Idéals pour les opérations numériques ou des calculs intensifs.
Opération courante	Flexible, totalement malléable et modifiable.	Possède moins de fonction que les listes, peu malléable.
Intégration	Compatibles avec la plupart des modules Python.	Recommandé pour des travaux scientifiques (Data, IA). Peu de compatibilité ailleurs.

Cas pratique

Reprenons ensemble le même exemple que celui des listes, mais adapté aux tableaux.

```
import array
# Création d'un tableau d'entiers
scores = array.array('i', [90, 85, 88, 92])
# Ajouter des éléments au tableau
scores.append(95)
print(scores) # Sortie : array('i', [90, 85, 88, 92, 95])
# Insérer un élément à une position spécifique
scores.insert(2,87)
print(scores) # Sortie : array('i', [98, 85, 87, 88, 92, 95])
# Supprimer un élément du tableau
scores.remove(85)
print(scores) # Sortie : array('i', [90, 87, 88, 92, 95])
#Inverser le tableau
scores.reverse()
print(scores) # Sortie : array('i', [95, 92, 88, 87, 90])
```

Tous droits réservés à STUDI - Reproduction interdite

José est un professeur qui souhaite suivre les notes de ses étudiants pour différents examens.

Il a besoin d'un système pour enregistrer, manipuler et analyser ces notes de manière efficace.

Il vous demande de concevoir un système capable de :

- Ajouter les notes des étudiants au fur et à mesure des examens
- Modifier les notes
- Supprimer des notes
- Calculer la moyenne des étudiants
- Trier les notes dans l'ordre croissant ou décroissant

Dans un premier temps, vous allez devoir concevoir votre tableau, avec un typage défini dès le début, ainsi qu'un jeu de données (supposons que les notes sont sur 100).

```
1 import array
2 #Tableau initial de notes
3 notes = array.array('i',[85, 90, 78, 92])
```

Pour ajouter et supprimer des notes, Python vous propose nativement append() et remove(), qui permettent respectivement d'ajouter et de supprimer des éléments dans un tableau.

```
1 import array
2 #Tableau initial de notes
3 notes = array.array('i', [85, 90, 78, 92])
4 #Ajouter des notes
5 notes.append(88)
6 notes.append(95)
7
8 #nous pouvons ajouter plusieurs notes à la fois !
9 notes.extend(array.array('i',[70, 65]))
10 print(notes) # array('i', [85, 90, 78, 92, 88, 95, 70, 65])
11 #Supprimer une note
12 notes.remove(78)
13 print(notes) # array('i', [85, 90, 92, 88, 95, 70, 65])
```

La modification est un peu plus complexe : vous devez connaître l'index de l'élément où José s'est trompé, puis, via cet indice, injecter une nouvelle valeur. Supposons qu'il souhaite modifier la note de 85 pour la remplacer par 90.

```
1 if 85 in notes: # Vérifie si 85 existe dans la liste
2 indice_erreur = notes.index(85)  # Trouve l'indice de la première occurrence de 85
3 notes[indice_erreur] = 90  # Remplace la valeur 85 par 90
4 print(notes)
5 else:
6 print("La valeur 85 n'existe pas dans la liste.")
```

Python propose deux méthodes natives permettant d'effectuer une moyenne et un tri, que ce soit pour une liste ou un tableau. Pour la moyenne, vous allez faire la somme de tous les éléments de votre tableau et diviser par la longueur de celui-ci.

```
1 # Calculer la novenne des notes
2 moyenne = sum(notes) / len(notes)
3 print(f"Moyenne des notes: {moyenne:.2f}") # Sortie : Moyenne des notes: 84.29
4 #Trier les notes
5 notes = sorted(notes)
6 print("Notes triées :", notes) # Sortie: Notes triées: [65, 70, 88, 90, 90, 92, 95]
```


Tous droits réservés à STUDI - Reproduction interdite

D. Les dictionnaires

Az Définition

Les dictionnaires en Python sont des collections non ordonnées de paires clé-valeur. Chaque élément d'un dictionnaire est une paire formée d'une clé unique et d'une valeur associée. Les clés doivent être immuables (par exemple, des chaînes de caractères, des nombres), tandis que les valeurs peuvent être de n'importe quel type de données.

Pour créer un dictionnaire, contrairement aux listes ou aux tableaux, il faut utiliser des accolades {} et séparer les clés et les valeurs par des deux-points :. Il est possible d'accéder aux valeurs en utilisant les clés correspondantes. Un dictionnaire peut avoir des clés de types différents et des valeurs de types différents. Cependant, il est recommandé de garder le même type de clé dans tout votre dictionnaire.

```
1 # Création d'un dictionnaire
2 etudiant = {
3     "nom": "Alice",
4     "age": 21,
5     "cours": ["Maths", "Physique", "Informatique"]
6 }
7
8 # Accès aux valeurs du dictionnaire
9 print(etudiant["nom"])  # Sortie : Alice
10 print(etudiant["cours"])  # Sortie : ['Maths', 'Physique', 'Informatique']
```

La différence fondamentale entre une liste et un dictionnaire est que le dictionnaire va stocker une structure complète (à l'image du format de données JSON). Pour accéder aux éléments, vous n'allez pas utiliser un indice, mais la clé, qui est unique. La valeur, quant à elle, peut être la même pour plusieurs clés.

De même que pour les listes ou les tableaux, les boucles peuvent être utilisées pour parcourir et manipuler les éléments d'un dictionnaire. Vous pouvez itérer sur les clés, les valeurs ou les paires clé-valeur.

```
1 #Boucle for pour parcourir les clés
2 for cle in etudiant.keys():
3     print(cle)
4
5 # Boucle for pour parcourir les valeurs
6 for valeur in etudiant.values():
7     print(valeur)
8
9 #Boucle for pour parcourir les paires clé-valeur
10 for cle, valeur in etudiant.items():
11   print(cle, ': ', valeur)
```

Fondamental

Les dictionnaires en Python possèdent plusieurs méthodes intégrées qui facilitent leur manipulation. Voici quelques-unes des méthodes les plus couramment utilisées :

- « *get(key)* » : renvoie la valeur associée à la clé si elle existe, sinon renvoie une valeur par défaut.
- « *keys()* » : renvoie une vue des clés du dictionnaire.
- « *values()* » : renvoie une vue des valeurs du dictionnaire.
- « *items()* » : renvoie une vue des paires clé-valeur du dictionnaire.

Tous droits réservés à STUDI - Reproduction interdite

- « *pop(key[, default])* » : supprime l'élément avec la clé spécifiée et renvoie sa valeur.
- « *update({ })* » : met à jour le dictionnaire avec les paires clé-valeur du dictionnaire en paramètre.

```
#Utilisation de la méthode get()
print(etudiant.get("nom")) #Sortie : Alice
print(etudiant.get("adresse", "Non spécifiée")) # Sortie : Men saisiti
# Utilisation de la méthode keys()
print(etudiant.keys()) # Sortie: dict_keys(['nom', 'age", 'cours'))
# Utilisation de la méthode values()
print(etudiant.values()) # Sortie : dict_values(['Alice', 21, ['Maths', 'Physique', 'Informatique'])
#Utilisation de la méthode items()
print(etudiant.items()) # Sortie : dict_items([('nom', 'Alice'), ('age', 21), ('cours', ['Maths', 'Physique', 'Informatique'])])
#* Utilisation de la méthode pop()
etudiant.pop("age")
print(etudiant) # Sortie : {'nom': 'Alice', 'cours': ['Maths', 'Physique', 'Informatique']}
#Utilisation de la méthode update()
etudiant.update({"age": 22, "adresse": "Paris"})
print(etudiant) # Sortie : {{'nom': 'Alice', 'cours': ['Maths', 'Physique', 'Informatique'], 'age': 22, 'adresse': 'Paris'}
```

Les dictionnaires possèdent également une méthode de nettoyage, cela permet de le vider.

```
# Nettoyer un dictionnaire
etudiant.clear()
print(etudiant)
```

Cas pratique

José est responsable des ressources humaines dans une entreprise et souhaite suivre les informations des employés de son département. Il a besoin d'un système pour gérer les informations des employés, telles que leurs noms, âges, postes et adresses.

Les besoins spécifiques de José sont les suivants :

1. Ajouter de nouveaux employés au système.
2. Si les informations d'un employé changent (comme une promotion ou un changement d'adresse), José doit pouvoir les mettre à jour.
3. Si un employé quitte l'entreprise, ses informations doivent être retirées.
4. José doit pouvoir lister toutes les informations des employés.

Vous allez maintenant implémenter un programme Python qui répond à ces besoins en utilisant les dictionnaires.

Dans un premier temps, il est important d'instancier et d'initialiser le dictionnaire.

```
# Dictionnaire initial d'employés
employes: dict = { #un dictionnaire est un dict en Python
   101: { "nom": "Alice", "age": 30, "poste": "Développeuse", "adresse": "Paris"},
   102: {"nom": "Bob", "age": 35, "poste": "Manager", "adresse": "Lyon"}
}
```

L'ajout d'une information dans un dictionnaire en python, se fait via l'utilisation de la méthode update().

```
# Dictionnaire initial d'employés
employes: dict = {  #un dictionnaire est un dict en Python
    101: {"nom": "Alice", "age": 30, "poste": "Développeur", "adresse": "Paris"},
    102: {"nom": "Bob", "age": 35, "poste": "Manager", "adresse": "Lyon"}
}

```


Tous droits réservés à STUDI - Reproduction interdite

```
7 employes.update({103: {"nom": "Charlie", "age": 28, "poste": "Designer", "adresse": "Marseille"}})
8 print(employes)
9 # {
10 # 101: {'nom': 'Alice', 'age': 30, 'poste': 'Développeur', 'adresse': 'Paris'},
11 # 102: {'nom': 'Bob', 'age': 35, 'poste': 'Manager', 'adresse': 'Lyon'},
12 # 103: {'nom': 'Charlie', 'age': 28, 'poste': 'Designer', 'adresse': 'Marseille'}
13 # }
```

Maintenant, vous souhaitez modifier l'adresse de l'employé 101. Il, ou elle, a déménagé sur Bordeaux. Pour ce faire, vous allez vérifier si la clé 101 existe, si oui, vous allez modifier la valeur.

```
1 if employes.get(101):
2     employes[101]["adresse"] = "Bordeaux"
3 print(employes[101])
4 # Sortie : {'nom': 'Alice', 'age': 30, 'poste': 'Développeur', 'adresse': 'Bordeaux'}
```

La suppression d'un élément peut s'effectuer via remove() ou pop(). Ici, vous allez utiliser pop(). pop() est utilisé lorsque vous souhaitez supprimer un élément d'un dictionnaire tout en récupérant sa valeur. Commencez par répondre au besoin de José :

```
1 # Supprimer un employé
2 employes.pop(102)
3 print(employes)
4 # {
5 # 101: {'nom': 'Alice', 'age': 30, 'poste': 'Développeur', 'adresse': 'Bordeaux'},
6 # 103: {'nom': 'Charlie', 'age': 28, 'poste': 'Designer', 'adresse': 'Marseille'}
7 # }
```

Ensuite, si vous affichez la suppression via pop() de la valeur détenue par la clé 102, vous effectuez une suppression, mais vous pouvez également voir que la valeur supprimée est retournée par l'exécution.

```
1 # Supprimer un employé
2 print(employes.pop(102)) # {'nom': 'Bob', 'age': 35, 'poste': 'Manager', 'adresse': 'Lyon'}
```

Enfin, pour afficher les données des employés, vous devez effectuer une boucle, parcourant les clés ainsi que les valeurs. Puis, les afficher via un print().

```
1 # Dictionnaire initial d'employés
2 employes: dict = {  #un dictionnaire est un dict en Python
3     101: {"nom": "Alice", "age": 38, "poste": "Développeur", "adresse": "Paris"},
4     102: {"nom": "Bob", "age": 35, "poste": "Manager", "adresse": "Lyon"}
5 }
6 # Afficher les informations de tous les employés
7 for id, infos in employes.items():
8     print(f"ID: {id}")
9     for cle, valeur in infos.items():
10         print(f"{cle}: {valeur}")
11
12 # Sortie :
13 # ID: 101
14 # nom: Alice
15 # age: 38
16 # poste: Développeur
17 # adresse: Paris
18 # ID: 102
19 # nom: Bob
20 # age: 35
21 # poste: Manager
22 # adresse: Lyon
```

Tous droits réservés à STUDI - Reproduction interdite

III Structure de données avancées

A. Les tuples

Az Définition

Les tuples en Python sont des collections ordonnées et immuables. Contrairement aux listes, une fois qu'un tuple est créé, ses éléments ne peuvent pas être modifiés, ajoutés ou supprimés. Les tuples sont définis en utilisant des parenthèses « *()* » et les éléments sont séparés par des virgules. Cette immuabilité rend les tuples utiles pour stocker des données qui ne doivent pas changer, offrant des garanties sur l'intégrité des données.

Pour créer un tuple, vous pouvez simplement placer une série d'éléments entre parenthèses. Vous pouvez accéder aux éléments d'un tuple en utilisant des indices, le premier élément ayant un indice de 0.

```
1 # Création d'un tuple
2 dimensions = (200, 50, 100)
3 # Accès aux éléments du tuple
4 print(dimensions[0]) # Sortie : 200
5 print(dimensions[1]) # Sortie : 50
6 print(dimensions [2]) # Sortie : 100
```

Il est également possible de vouloir obtenir plusieurs valeurs contenues dans le tuple !

```
1 # Création d'un tuple
2 dimensions = (200, 50, 100)
3 # Accès aux éléments du tuple
4 print(dimensions[0:2]) # Sortie : (200, 50)
5 print(dimensions[1]) # Sortie : 50
6 print(dimensions [2]) # Sortie : 100
```

Vous pouvez accéder aux éléments via les boucles, comme les autres structures de données.

```
1 # Création d'un tuple
2 dimensions = (200, 50, 100)
3 # Accès aux éléments du tuple
4 print(dimensions[0:2])  # Sortie : (200, 50)
5 print(dimensions[1])  # Sortie : 50
6 print(dimensions[2])  # Sortie : 100
7
8 for dimension in dimensions:
9     print(dimension)
10
11 i = 0
12 while i < len(dimensions):
13     print(dimensions[i])
14     i += 1
```

Parcourir un tuple est très similaire au parcours des listes, mais les éléments dans le tuple ne sont pas modifiables. En effet, ils sont souvent utilisés pour stocker des collections de données hétérogènes, telles que des enregistrements d'une base de données ou des structures de données composées. Étant donné que les tuples sont immuables, ils peuvent être utilisés comme clés dans des dictionnaires, ce qui n'est pas possible avec les listes.

Essayons ensemble de mettre un tuple comme clé d'un dictionnaire.

Tous droits réservés à STUDI - Reproduction interdite

```
1 # Utilisation des tuples comme clés de dictionnaire
2 points = {
3 (0, 0): "Origine",
4 (1, 2): "Point A",
5 (3, 4): "Point B"
6 }
7 print(points[(1, 2)]) # Sortie : Point A
```

Comme vous l'avez déjà vu, vous pouvez mettre n'importe quel type en clé, mais, une fois que vous avez défini le type, il est recommandé de ne pas le modifier.

B. Les piles

Az Définition

Les piles (stacks) en Python sont des structures de données ordonnées qui suivent le principe LIFO (Last In, First Out). Cela signifie que le dernier élément ajouté à la pile sera le premier à être retiré. Les piles sont couramment utilisées dans les algorithmes et les programmes pour gérer des tâches comme l'annulation d'actions, le parcours de structures de données et la gestion des récursions.

En Python, les piles peuvent être implémentées en utilisant des listes ou le module collections.deque. L'utilisation des listes est simple et efficace pour des piles de petite à moyenne taille, tandis que deque offre des performances optimales pour les opérations sur des piles plus volumineuses.

```
1 # Implémentation d'une pile avec une liste
2 pile = []
3 # Opération push (ajouter un élément) pile.append(1)
4 pile.append(2)
5 pile.append(3)
6 print(pile) # Sortie :[2, 3]
7 # Opération pop (retirer un élément)
8 element = pile.pop()
9 print(element) # Sortie : 3
10 print(pile) # Sortie : [2]
```

Les boucles peuvent être utilisées pour parcourir les éléments d'une pile, mais en pratique, les piles sont souvent manipulées en utilisant les opérations push et pop pour ajouter et retirer des éléments.

```
1 element = pile.pop()
2 print(element) # Sortie : 3
3 print(pile) # Sortie : [2]
4
5 # parcourir une pile
6 for element in pile:
7     print(element)
8
9 #vider une pile
10 while pile:
11     print(pile.pop())
```

Les piles sont utilisées dans de nombreux algorithmes et structures de données. Elles sont particulièrement utiles pour gérer les opérations récursives, les parcours de graphes, les algorithmes de backtracking et les systèmes d'annulation/redémarrage.

Tous droits réservés à STUDI - Reproduction interdite

```python
# Vérification de la validité des parenthèses dans une expression
expression = "(1 + (2 * 3) + (4 / 2))"
pile = []
for char in expression:
    if char == '(':
        pile.append(char)
    elif char == ')':
        if not pile:
                print('False')
        pile.pop()
print(not pile)
```

C. Les files

Az Définition

Les files (queues) en Python sont des structures de données ordonnées qui suivent le principe FIFO (First In, First Out). Cela signifie que le premier élément ajouté à la file sera le premier à être retiré. Les files sont couramment utilisées dans les algorithmes et les programmes pour gérer des tâches telles que la planification des tâches, la gestion des files d'attente et la coordination des processus.

En Python, les files peuvent être implémentées en utilisant des listes, le module « *collections.deque* » ou le module « *queue* ». L'utilisation des listes est simple mais inefficace pour les grandes files, car les opérations de suppression en début de liste sont coûteuses. Le module « *deque* » de « *collections* » offre des performances optimales pour les opérations de file. Le module « *queue* » est particulièrement utile pour les programmes multi-threading.

```python
# Implémentation d'une file avec une liste
file = []
# Opération enqueue (ajouter un élément)
file.append(1)
file.append(2)
file.append(3)
print(file) # Sortie : [1, 2, 3]

#Opération dequeue (retirer un élément)
element = file.pop(0)
print(element) # Sortie : 1
print(file) # Sortie : [2, 3]
```

Les boucles peuvent être utilisées pour parcourir les éléments d'une file, mais en pratique, les files sont souvent manipulées en utilisant les opérations enqueue et dequeue pour ajouter et retirer des éléments.

```python
# Implémentation d'une file avec une liste
file = []
# Opération enqueue (ajouter un élément)
file.append(1)
file.append(2)
file.append(3)

# parcourir les elements d'une file
for element in file:
    print(element)

i = 0
# vider une file
while file:
    print(file.pop(0))
```


Tous droits réservés à STUDI - Reproduction interdite

Les files sont utilisées dans de nombreux algorithmes et structures de données. Elles sont particulièrement utiles pour gérer les files d'attente, les parcours en largeur (BFS) dans les graphes et les tâches de planification.

Prenons l'exemple d'une caisse de supermarché.

```
1 from collections import deque
2
3 file_attente = deque()
4
5 # Ajouter des clients à la file
6 file_attente.append("Alice")
7 file_attente.append("Bob")
8 file_attente.append("Charlie")
9
10 print("Clients en attente :", list(file_attente))
11 # Sortie: Clients en attente: ['Alice', 'Bob', 'Charlie']
12
13 # Servir un client
14 client = file_attente.popleft()
15 print(f"{client} est servi.") # Sortie : Alice est servi.
16 print("Clients en attente :", list(file_attente)) # Sortie: Clients en attente :
['Bob', 'Charlie']
```

Vous avez trois personnes dans votre file, Alice, Bob et Charlie.

Au fur et à mesure, vous allez les traiter un à un, donc les faire sortir de votre file. Prenons ici l'exemple d'Alice.

Tous droits réservés à STUDI - Reproduction interdite

IV Structures spécialisées

A. Les graphes

Az Définition

Les graphes sont des structures de données qui modélisent les relations entre des objets. Ils se composent de nœuds (ou sommets) et d'arêtes (ou liens) qui relient ces nœuds. Les graphes sont utilisés pour représenter des réseaux complexes comme les réseaux de transport, les réseaux sociaux ou encore les liens hypertextes entre pages web.

Il existe plusieurs types de graphes :

1. **Graphes orientés** : les arêtes ont une direction, indiquant une relation unidirectionnelle entre les nœuds (par exemple, suivre une direction d'une ville à une autre).
2. **Graphes non orientés** : les arêtes n'ont pas de direction, indiquant une relation bidirectionnelle entre les nœuds (par exemple, une amitié sur un réseau social).
3. **Graphes pondérés** : les arêtes ont des poids, indiquant un coût ou une distance associée à la relation (par exemple, la distance entre deux villes).

Il existe plusieurs façons de représenter les graphes en programmation. Les deux plus courantes sont les listes d'adjacence et les matrices d'adjacence. Une liste d'adjacence est un tableau de listes où l'entrée à chaque index représente un nœud et contient une liste des nœuds adjacents (connectés).

```
1 # Représentation d'un graphe non orienté
2 graphe = {
3     'A': ['B', 'C'],
4     'B': ['A', 'D', 'E'],
5     'C': ['A', 'F'],
6     'D': ['B'],
7     'E': ['B', 'F'],
8     'F': ['C', 'E']
9 }
```

Dans cet exemple, le nœud « *A* » est connecté aux nœuds « *B* » et « *C* », le nœud « *B* » est connecté aux nœuds « *A* », « *D* » et « *E* », et ainsi de suite.

La représentation la plus simple d'un Graphe est l'algorithme de Dijkstra. L'algorithme de Dijkstra trouve le chemin le plus court d'un nœud de départ vers tous les autres nœuds dans un graphe pondéré. C'est un algorithme gourmand qui utilise une approche systématique pour toujours choisir le nœud le plus proche non encore traité.

Voyons ensemble comment il est constitué, tout d'abord initialisons notre graphe.

```
1 graphe = {
2     'A': {'B': 1, 'C': 4},
3     'B': {'A': 1, 'D': 2, 'E': 5},
4     'C': {'A': 4, 'F': 3},
5     'D': {'B': 2},
6     'E': {'B': 5, 'F': 1},
7     'F': {'C': 3, 'E': 1}
8 }
```

Un graphe pondéré est un graphe où chaque arête a un poids associé, représentant un coût, une distance ou une autre mesure entre les nœuds connectés. Par exemple, le nœud A est connecté au nœud B avec un point de 1, le nœud est également connecté au nœud C avec un poids de 4.

Tous droits réservés à STUDI - Reproduction interdite

Cette représentation est beaucoup utilisée, par exemple, si ce graphe pondéré représente le nombre de minutes que nous devons marcher, cela voudrait dire que je dois marcher 1 minutes pour aller de mon point A à mon poids B, et 4 minutes, pour aller de mon point A au point C.

Effectuons une première utilisation de l'algorithme de Dijkstra.

```
graphe = {
    'A': {'B': 1, 'C': 4},
    'B': {'A': 1, 'D': 2, 'E': 5},
    'C': {'A': 4, 'F': 3},
    'D': {'B': 2},
    'E': {'B': 5, 'F': 1},
    'F': {'C': 3, 'E': 1}
}


def dijkstra(graphe, start):
    # Initialisation des distances avec l'infini
    distances = {node: float('infinity') for node in graphe}
    distances[start] = 8
    #Initialisation des noeuds visit
    to_visit = set(graphe.keys())

    while to_visit:
        #Sélectionner le noeud avec la plus petite distance
        current_node = min(to_visit, key=lambda node: distances[node])
        to_visit.remove(current_node)
        #Mise à jour des distances des voisins
        for voisin, poids in graphe[current_node].items():
            distance = distances[current_node] + poids
            if distance < distances[voisin]:
                distances[voisin] = distance
    return distances


distances_from_a = dijkstra(graphe, "A")
print("Distances à partir de A :", distances_from_a)
# Distances à partir de A : {'A': 8, 'B': 9, 'C': 12, 'D': 11, 'E': 14, 'F': 15}
```

- Initialisation : initialise toutes les distances à l'infini sauf pour le nœud de départ qui est à zéro. Utilise un ensemble pour garder les nœuds à visiter.
- Traitement : tant qu'il y a des nœuds à visiter, sélectionne le nœud avec la plus petite distance actuelle. Pour chaque voisin, calcule la nouvelle distance et si elle est plus petite que la distance connue, met à jour la distance.

Par exemple, pour aller de A à D, vous devez passer par B. Donc vous devez parcourir un poids de 1, pour aller de A à B, ensuite un poids de 2 pour aller de B à D, donc cela donne 3.

Vous pouvez optimiser votre approche en proposant de chercher le chemin le plus court entre deux nœuds toujours en utilisant l'algorithme de Dijkstra.

```
def trouver_chemin_le_plus_court(graphe, start, end):
    distances = dijkstra(graphe, start)
    chemin = []
    current_node = end
    while current_node != start:
        for voisin, poids in graphe[current_node].items():
            if distances[current_node] - poids == distances[voisin]:
                chemin.append(current_node)
                current_node = voisin
                break
    chemin.append(start)
    chemin.reverse()
    return chemin
```

Tous droits réservés à STUDI - Reproduction interdite

```


chemin = trouver_chemin_le_plus_court(graphe, 'A', 'F')
print("Chemin le plus court de A à F :", chemin)  # Sortie : Chemin le plus court de A à F : ['A', 'C', 'F']
```

Le but de ce code est de trouver le chemin le plus rapide entre A et F. Pour cela, vous avez utilisé votre algorithme de Dijkstra dans le but d'obtenir tous les nœuds entre le début et la fin, que vous pouvez exploiter.

Votre approche est à l'envers, c'est-à-dire, que vous allez partir de F, regarder les poids, puis, prendre constamment le plus court.

Fondamental

Les graphes sont des structures de données puissantes pour modéliser des réseaux complexes. Les algorithmes de parcours en largeur (BFS) et en profondeur (DFS) permettent d'explorer les graphes de manière systématique, tandis que l'algorithme de Dijkstra offre une solution efficace pour trouver les chemins les plus courts dans les graphes pondérés.

B. Les arbres

Méthode

Les arbres sont des structures de données hiérarchiques composées de nœuds, où chaque nœud contient des données et peut avoir des nœuds enfants. Le nœud supérieur est appelé racine, et chaque nœud, sauf la racine, a exactement un parent. Les arbres n'ont pas de cycles et sont utilisés pour représenter des structures hiérarchiques telles que les systèmes de fichiers, les structures d'organisation, etc.

Il existe deux types d'arbre :

1. **Arbre binaire** : un arbre dans lequel chaque nœud a au plus deux enfants, appelés nœud gauche et nœud droit.
2. **Arbre binaire de recherche (BST)** : un arbre binaire où chaque nœud a un nœud gauche avec des valeurs plus petites que le nœud lui-même et un nœud droit avec des valeurs plus grandes. Cela permet une recherche efficace des éléments.

Pour avoir un exemple de ces deux types d'arbres, vous allez avoir besoin d'une structure.

```
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
```

Ensuite, insérez vos éléments :

```
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key


# Création des nœuds de l'arbre
root = TreeNode(1)
root.left = TreeNode(2)
```


Tous droits réservés à STUDI - Reproduction interdite

```
11 root.right = TreeNode(3)
12 root.left.left = TreeNode(4)
13 root.left.right = TreeNode(5)
14 root.right.left = TreeNode(6)
15 root.right.right = TreeNode(7)
16 # Parcours en pré-ordre (root-left-right)
17
18 def pre_order(node):
19     if node:
20         print(node.val, end=" ")
21         pre_order(node.left)
22         pre_order(node.right)
23 print("Parcours en pré-ordre de l'arbre binaire :")
24 pre_order(root) # Sortie : 1 2 4 5 3 6 7
```

Dans cet exemple, vous avez saisi les éléments à droite ou à gauche, sans aucune distinction, de manière totalement arbitraire. Le BST (Binary Search Tree) fonctionne de manière similaire, mais à gauche vous aurez toujours les valeurs inférieures au nœud actuel, et à droite, toujours des valeurs supérieures.

Vous avez besoin d'une fonction logique qui va « *réfléchir* » en fonction de la valeur et la placer correctement dans l'arbre :

```
1 def insert(root, key):
2     if root is None:
3         return TreeNode(key)
4     else:
5         if root.val < key:
6             root.right = insert(root.right, key)
7         else:
8             root.left = insert(root.left, key)
9     return root
10
11
12 root = TreeNode(50)
13 root = insert(root, 30)
14 root = insert(root, 20)
15 root = insert(root, 40)
16 root = insert(root, 70)
17 root = insert(root, 60)
18 root = insert(root, 80)
```

Enfin, procédez à l'affichage des données du BST. L'approche est plus complexe mais les résultats sont plus pertinents et performants.

```
1 # Parcours en en-ordre (left-root-right) du BST
2 def in_order(node):
3     if node:
4         in_order(node.left)
5         print(node.val, end=" ")
6         in_order(node.right)
7
8
9 print("Parcours en en-ordre de l'arbre BST :")
10 in_order(root)  # Sortie : 20 30 40 50 60 70 80
```

Au même titre que l'insertion, vous pouvez supprimer des éléments, mais il faut faire attention à deux cas : le nœud a un seul enfant ou le nœud a deux enfants.

Cela peut vite devenir compliqué, pour cela, Python possède un module : anytree. Il permet une gestion plus simple de vos arbres.

Voici comment, avec anytree, vous pouvez concevoir un arbre et y afficher les données.

Tous droits réservés à STUDI - Reproduction interdite

```
1 from anytree import Node, RenderTree
2
3 #Création de l'arbre
4 root = Node("Racine")
5 child_un = Node("Enfant 1", parent=root)
6 child_deux = Node("Enfant 2", parent=root)
7 child_un_un = Node("Enfant 1.1", parent=child_un)
8 child_un_deux = Node("Enfant 1.2", parent=child_un)
9 child_deux_un = Node("Enfant 2.1", parent=child_deux)
10 # Affichage de l'arbre
11 for pre, fill, node in RenderTree(root):
12     print("%s%s" % (pre, node.name))
13 # Racine
14 # ├── Enfant 1
15 # │   ├── Enfant 1.1
16 # │   └── Enfant 1.2
17 # └── Enfant 2
18 #     └── Enfant 2.1
```

Auparavant, vous avez dû concevoir plusieurs méthodes pour explorer votre arbre, cela peut s'avérer extrêmement complexe en fonction de la logique que vous souhaitez mettre en arbre. Mais heureusement, anytree possède un module « *search()* ».

```
1 from anytree import Node, RenderTree, find_by_attr
2
3 #Création de l'arbre
4 root = Node("Racine")
5 child_un = Node("Enfant 1", parent=root)
6 child_deux = Node("Enfant 2", parent=root)
7 child_un_un = Node("Enfant 1.1", parent=child_un)
8 child_un_deux = Node("Enfant 1.2", parent=child_un)
9 child_deux_un = Node("Enfant 2.1", parent=child_deux)
10
11 # Affichage de l'arbre
12 node = find_by_attr(root,name="name",value="Enfant 1.1")
13 print(node)
14
15 new_child = Node("nouveau enfant",parent=node)
16 for pre, fill, node in RenderTree(root):
17     print("%s%s" % (pre, node.name))
18 #Racine
19 #├── Enfant 1
20 #│   ├── Enfant 1.1
21 #│   │   └── nouveau enfant
22 #│   └── Enfant 1.2
23 #└── Enfant 2
24 #    └── Enfant 2.1
```

Le seul défaut que possède ce module c'est qu'il ne gère pas directement la suppression, vous devez positionner le nœud à « *None* » afin de supprimer tous ses enfants.

```
1 from anytree import Node, RenderTree, find_by_attr
2
3 #Création de l'arbre
4 root = Node("Racine")
5 child_un = Node("Enfant 1", parent=root)
6 child_deux = Node("Enfant 2", parent=root)
7 child_un_un = Node("Enfant 1.1", parent=child_un)
8 child_un_deux = Node("Enfant 1.2", parent=child_un)
9 child_deux_un = Node("Enfant 2.1", parent=child_deux)
10
11 # Suppression d'un noeud en changeant le parent de ses enfants
12 child_un_un.parent = root
13 child_un.parent = None # ceci enleve le noeud 'Enfant 1' et ses enfants
```


Tous droits réservés à STUDI - Reproduction interdite

```
14
15
16 for pre, fill, node in RenderTree(root):
17     print("%s%s" % (pre, node.name))
18 #Racine
19 #├── Enfant 2
20 #│   └── Enfant 2.1
21 #└── Enfant 1.1
```

Tous droits réservés à STUDI - Reproduction interdite

V Essentiel

Les structures de données sont fondamentales pour organiser et manipuler les données efficacement en Python. Parmi les structures de base, on trouve les listes, qui sont des collections ordonnées d'éléments modifiables, et les tableaux (arrays), utiles pour les opérations mathématiques optimisées. Les dictionnaires sont des collections non ordonnées de paires clé-valeur, permettant un accès rapide aux données via des clés uniques.

Pour des structures de données plus avancées, les tuples sont similaires aux listes, mais immuables, offrant une utilisation sécurisée pour des données constantes. Les piles suivent le principe LIFO (Last In, First Out), tandis que les files suivent le principe FIFO (First In, First Out), tous deux étant utiles pour des opérations séquentielles spécifiques.

Les arbres, notamment les arbres binaires et les arbres binaires de recherche (BST), organisent les données de manière hiérarchique, facilitant les opérations de recherche, d'insertion et de suppression. Les graphes modélisent les relations complexes entre objets, avec des applications dans les réseaux de transport et les réseaux sociaux. Les parcours de graphes (BFS, DFS) permettent d'explorer ces structures, tandis que l'algorithme de Dijkstra est utilisé pour trouver les chemins les plus courts dans les graphes pondérés.

Tous droits réservés à STUDI - Reproduction interdite

VI Auto-évaluation

A. Exercice

Question 1

[solution n°1 p. 29]

Créez un dictionnaire pour gérer un stock de livres. Chaque livre a un identifiant unique (clé) et une quantité disponible (valeur). Ajoutez plusieurs livres, modifiez la quantité d'un livre spécifique, supprimez un livre du stock, et affichez le stock complet avec les identifiants et les quantités.

Question 2

[solution n°2 p. 29]

Utilisation des tuples et des arbres. Créez un tuple pour stocker les coordonnées (x, y, z) d'un point dans l'espace. Ensuite, créez un arbre binaire de recherche (BST) pour stocker et trier ces points selon leur distance à l'origine (0, 0, 0). Implémentez l'insertion et l'affichage des points dans l'ordre croissant de leur distance.

B. Test

Exercice 1 : Quiz

[solution n°3 p. 30]

Question 1

Quelle est la principale différence entre une liste et un tableau en Python ?

- Les listes peuvent contenir différents types de données, à l'exception des tableaux
- Les tableaux sont plus rapides que les listes
- Les listes sont immuables, les tableaux sont modifiables
- Les listes nécessitent une déclaration de type, à l'exception des tableaux

Question 2

Quelle méthode est utilisée pour ajouter un élément à la fin d'une liste ?

- append()
- insert()
- extend()
- push()

Question 3

Quelle méthode de liste est utilisée pour supprimer et renvoyer l'élément à une position donnée ?

- remove()

Tous droits réservés à STUDI - Reproduction interdite

delete()

pop()

discard()

Question 4

Comment accéder au dernier élément d'une liste en utilisant un indice négatif ?

list[-1]

list[0]

list[len(list)]

list[-0]

Question 5

Quelle structure de données est la plus adaptée pour modéliser une relation clé-valeur en Python ?

Liste

Tableau

Dictionnaire

Tuple

Question 6

Quelle est la caractéristique principale d'un tuple en Python ?

Il est modifiable

Il est immuable

Il ne peut contenir que des nombres

Il est toujours trié

Tous droits réservés à STUDI - Reproduction interdite

Question 7

Quelle est la différence entre une pile (stack) et une file (queue) ?

- Une pile suit le principe FIFO, tandis qu'une file suit le principe LIFO
- Une pile suit le principe LIFO, tandis qu'une file suit le principe FIFO
- Les piles sont ordonnées, les files non
- Les files sont immuables, les piles sont modifiables

Question 8

Quelle structure de données permet de représenter efficacement des réseaux comme les réseaux de transport ou les réseaux sociaux ?

- Les graphes
- Les arbres
- Les piles
- Les dictionnaires

Question 9

Quelle méthode d'un dictionnaire permet de renvoyer toutes les paires clé-valeur ?

- items()
- keys()
- values()
- pairs()

Question 10

Quelle est la différence principale entre un graphe orienté et un graphe non orienté ?

- Les arêtes d'un graphe orienté ont une direction
- Les arêtes d'un graphe orienté n'ont pas de direction
- Un graphe orienté ne peut pas avoir de cycles

Tous droits réservés à STUDI - Reproduction interdite

Les nœuds d'un graphe orienté sont immuables

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

[exercice p. 25] Solution n°1

```
1 # Création du dictionnaire pour le stock de livres
2 stock_livres = {
3     101: 5,  # Identifiant du livre 101 avec 5 exemplaires en stock
4     102: 2,
5     103: 8
6 }
7
8 # Ajouter des livres au stock
9 stock_livres[104] = 10  # Ajout d'un livre avec l'identifiant 104 et 10 exemplaires
10 stock_livres[105] = 7
11
12 # Modifier la quantité d'un livre spécifique
13 stock_livres[102] = 4  # Modification de la quantité du livre avec l'identifiant 102
à 4 exemplaires
14
15 # Supprimer un livre du stock
16 del stock_livres[103]  # Suppression du livre avec l'identifiant 103
17
18 # Afficher le stock complet
19 for id_livre, quantite in stock_livres.items():
20     print(f"Livre ID: {id_livre}, Quantité en stock: {quantite}")
```

Explication :

- **Création du dictionnaire** : le dictionnaire stock_livres est créé avec des identifiants de livres comme clés et des quantités disponibles comme valeurs.
- **Ajout de nouveaux livres** : utilisation de l'opérateur d'affectation (=) pour ajouter de nouvelles paires clé-valeur au dictionnaire.
- **Modification d'un livre** : la quantité d'un livre spécifique est modifiée en accédant à la clé correspondante.
- **Suppression d'un livre** : le livre avec l'identifiant 103 est supprimé du dictionnaire avec l'instruction del.
- **Affichage du stock** : la boucle for itère sur les paires clé-valeur du dictionnaire pour afficher le stock actuel.

[exercice p. 25] Solution n°2

```
1 import math
2
3 # Classe pour représenter un nœud de l'arbre binaire de recherche
4 class TreeNode:
5     def __init__(self, point):
6         self.left = None
7         self.right = None
8         self.point = point
9         self.distance = math.sqrt(point[0]**2 + point[1]**2 + point[2]**2)
10
11 # Fonction pour insérer un point dans l'arbre binaire de recherche
12 def insert(root, point):
13     if root is None:
14         return TreeNode(point)
```

Tous droits réservés à STUDI - Reproduction interdite

```python
    else:
        # Calcul de la distance du point à l'origine
        distance = math.sqrt(point[0]**2 + point[1]**2 + point[2]**2)
        if distance < root.distance:
            root.left = insert(root.left, point)
        else:
            root.right = insert(root.right, point)
    return root

# Fonction pour afficher les points dans l'ordre croissant de leur distance
def in_order(node):
    if node:
        in_order(node.left)
        print(f"Point: {node.point}, Distance à l'origine: {node.distance:.2f}")
        in_order(node.right)

# Création et insertion des points dans le BST
points = [(1, 2, 2), (3, 4, 5), (0, 0, 0), (2, 1, 1), (6, 8, 9)]
root = None

for point in points:
    root = insert(root, point)

# Affichage des points dans l'ordre croissant de leur distance
print("Points triés par distance à l'origine :")
in_order(root)
```

Explication :

- **Classe TreeNode** : représente chaque nœud de l'arbre. Chaque nœud contient un point (sous forme de tuple) et la distance de ce point à l'origine.
- **Fonction insert** : cette fonction insère un point dans l'arbre en fonction de sa distance à l'origine. Les points avec une distance inférieure vont à gauche, et ceux avec une distance supérieure vont à droite.
- **Fonction in_order** : permet de parcourir l'arbre et d'afficher les points dans l'ordre croissant de leur distance à l'origine.
- **Insertion des points** : les points sont insérés dans l'arbre en utilisant la fonction insert.
- **Affichage des points triés** : la fonction in_order affiche les points dans l'ordre croissant de leur distance.

Solution n°3

[exercice p. 25]

Question 1

Quelle est la principale différence entre une liste et un tableau en Python ?

- ✓ Les listes peuvent contenir différents types de données, à l'exception des tableaux
- Les tableaux sont plus rapides que les listes
- Les listes sont immuables, les tableaux sont modifiables
- Les listes nécessitent une déclaration de type, à l'exception des tableaux

Tous droits réservés à STUDI - Reproduction interdite

Les listes peuvent contenir différents types de données alors que les tableaux nécessitent un type unique de données.

Question 2

Quelle méthode est utilisée pour ajouter un élément à la fin d'une liste ?

- ✓ append()
- insert()
- extend()
- push()

La méthode « *append()* » ajoute un élément à la fin de la liste.

Question 3

Quelle méthode de liste est utilisée pour supprimer et renvoyer l'élément à une position donnée ?

- remove()
- delete()
- ✓ pop()
- discard()

La méthode « *pop([i])* » supprime l'élément à la position donnée et le renvoie.

Question 4

Comment accéder au dernier élément d'une liste en utilisant un indice négatif ?

- ✓ list[-1]
- list[0]
- list[len(list)]
- list[-0]

Tous droits réservés à STUDI - Reproduction interdite

En Python, « *list[-1]* » permet d'accéder au dernier élément de la liste.

Question 5

Quelle structure de données est la plus adaptée pour modéliser une relation clé-valeur en Python ?

- Liste
- Tableau
- ✓ Dictionnaire
- Tuple

Les dictionnaires sont utilisés pour stocker des paires clé-valeur.

Question 6

Quelle est la caractéristique principale d'un tuple en Python ?

- Il est modifiable
- ✓ Il est immuable
- Il ne peut contenir que des nombres
- Il est toujours trié

Les tuples sont immuables, ce qui signifie qu'une fois créés, leurs éléments ne peuvent pas être modifiés.

Question 7

Quelle est la différence entre une pile (stack) et une file (queue) ?

- Une pile suit le principe FIFO, tandis qu'une file suit le principe LIFO
- ✓ Une pile suit le principe LIFO, tandis qu'une file suit le principe FIFO
- Les piles sont ordonnées, les files non

Tous droits réservés à STUDI - Reproduction interdite

Les files sont immuables, les piles sont modifiables

Les piles suivent le principe LIFO (Last In, First Out) et les files suivent le principe FIFO (First In, First Out).

Question 8

Quelle structure de données permet de représenter efficacement des réseaux comme les réseaux de transport ou les réseaux sociaux ?

- ✓ Les graphes
- Les arbres
- Les piles
- Les dictionnaires

Les graphes modélisent les relations entre des objets, comme dans les réseaux de transport.

Question 9

Quelle méthode d'un dictionnaire permet de renvoyer toutes les paires clé-valeur ?

- ✓ items()
- keys()
- values()
- pairs()

La méthode « *items()* » renvoie une vue des paires clé-valeur du dictionnaire.

Question 10

Quelle est la différence principale entre un graphe orienté et un graphe non orienté ?

- ✓ Les arêtes d'un graphe orienté ont une direction
- Les arêtes d'un graphe orienté n'ont pas de direction

Tous droits réservés à STUDI - Reproduction interdite

Un graphe orienté ne peut pas avoir de cycles

Les nœuds d'un graphe orienté sont immuables

Dans un graphe orienté, les arêtes ont une direction, indiquant une relation unidirectionnelle entre les nœuds.

Tous droits réservés à STUDI - Reproduction interdite